When Does a Reader-Side Lifetime Fix Help? Admission Coupling in an LSM Engine

V V Ramarathinam Iyer
Department of Database Systems, School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu, India
Email: ram.iyer200722@gmail.com

***Abstract*—A read lock held across a lookup easily keeps a storage file alive, but a reader-side fix cannot shorten the writer's critical section. We ask when such a fix helps. In seven pre-registered campaigns on one LSM engine, we compare a coarse read lock, per-table leases, and immutable reader sets across write load, compaction, and table count. With a saturated writer, the coarse lock completed up to 69% fewer writes as its mean hold grew to about 1.9 ms; below capacity, write throughput matched while median write response rose up to 2.66 times. Readers and writers are coupled through lock admission: each exclusive acquisition admits a round of readers. An intervention removing a redundant second acquisition supported this and removed the coarse lock's read advantage at injected delays up to 20 μs. The writer also limits both replacements, which still take the manager lock to search memory: at a high write rate it held the lock for about 72% of the window, about 1 ms per commit, and reads fell as that hold grew, though not in proportion as predicted. Leases and reader sets matched on throughput within a declared margin across publication and compaction, but read-only load separated them at high table counts, with leases completing 34% fewer reads at 256 tables and sixteen readers; their read equivalence was not established at high write rates. Results come from one engine on one Windows host, at most one writer, injected delay, and WAL fsync under the exclusive lock.**

***Index Terms*—LSM tree, reader-writer lock, descriptor leasing, immutable snapshots, controlled experiment.**

I. Introduction

A reader-side lifetime fix shortens the reader's lock hold but leaves the writer's own critical section in place. An LSM engine buffers updates in memory and writes sorted runs to disk [1], [2]; readers need stable file references while flushes and compactions replace those runs. Holding the manager's read lock across a lookup gives that protection, but exclusive acquisitions then wait for the lookup, including file I/O. Production engines instead retain references after releasing a short coordination lock [3]–[5]. We ask when such a fix helps, given the writer's own critical section. Our answer is that the writer decides the outcome, through how many times it acquires the lock and how long it holds it.

We compare CoarseLock, which retains the manager lock, with per-table leases (Lease) and immutable reader sets (Snapshot) in one Go engine. Injected delay controls lookup cost, and the writer's validation and WAL synchronization are shared. The paper makes three contributions:

- **What holding the lock costs.** The cost depends on how long the lock is held and on whether the writer is at capacity: a saturated writer loses throughput, while below capacity throughput matches and the cost appears in median write response (Section IV).
- **Admission coupling.** Each exclusive acquisition in a commit admits a round of readers. The coarse lock's read gain at short injected delays depends on the redundant second acquisition, and removing that acquisition removed the gain (Section V).
- **When Lease and Snapshot differ.** They matched on throughput across publication and compaction, and separated under read-only load at high table counts. Their read equivalence was not established at high write rates (Section VI).

The writer's own hold also limits both replacements (Section VII), and a trial-spacing lesson affects how such measurements should be run (Section VIII).

II. Background and Designs

A. Lifetime and admission

Stable references let a reader outlive a short coordination critical section. LevelDB takes references on memtables and the current version under its mutex and releases it while reading; its writer also releases the mutex around log append and synchronization [3]. RocksDB readers reference a SuperVersion, and Pebble readers a read state [4], [5]. Reader-writer locks admit concurrent readers while excluding writers [12]. Go's `RWMutex` blocks new readers once a writer waits and releases queued readers on writer unlock [18], so a continually waiting writer alternates with admitted readers.

B. Three designs

All designs share a skip-list memtable [11], SSTables with 4 KiB data blocks and a flat sparse index, and one `ReadAt` per visited block; there is no application block cache. Every read first searches active and frozen memory under `Mu.RLock`. CoarseLock keeps that lock until the SSTable search ends. Lease copies table-reader pointers under the lock, takes an atomic lease on every active table, and searches outside the lock, even when only one range can match. Snapshot pins one immutable reader-set descriptor under the manager read lock and `snapshotMu`, and searches outside both. Both replacements still take `Mu.RLock` to search memory. Table I summarizes the designs.

TABLE I
LIFETIME DESIGNS WITH THE DEFAULT COMMIT PATH.

Property	CoarseLock	Lease	Snapshot
Memory-tier search under `Mu.RLock`	yes	yes	yes
Reader SSTable I/O under `Mu.RLock`	yes	no	no
Disk-reader lifetime	lock	table leases	set pin
Per-read lifetime synchronization	`Mu`	atomic leases	`snapshotMu`
New set descriptor on publication	no	no	yes
Conflict check and WAL sync under `Mu.Lock`	yes	yes	yes
Exclusive acquisitions per successful commit	2	2	2

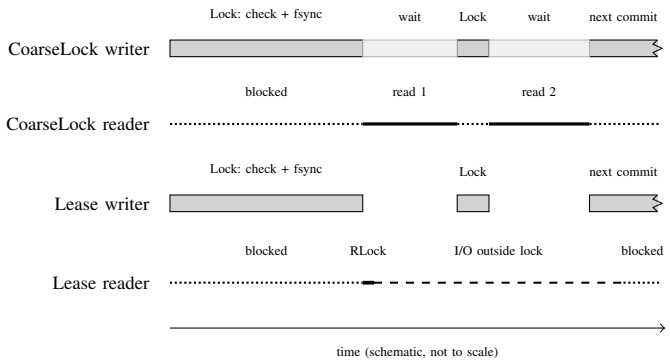


Fig. 1. One commit cycle under CoarseLock and Lease (schematic, not a measured trace). Under CoarseLock each reader completes one read per exclusive acquisition. Under Lease the reader's I/O is outside the lock and can outlast the short second acquisition.

C. The commit path

The default commit path gives every design two exclusive acquisitions per successful commit, so $k = 2$. `Transaction.Commit` takes `Mu.Lock`, validates conflicts, appends and synchronizes the WAL, updates memory, and unlocks. It then calls `checkAndTriggerFlush`, which reacquires the exclusive lock even when no flush is needed. That second acquisition is redundant without a flush; Section V-B removes it. Validation can read delayed SSTable blocks, and validation and fsync stay inside the exclusive hold after either reader-side fix. Each acquisition is a separate opportunity to admit queued readers (Fig. 1).

III. METHOD

The host is an AMD Ryzen 7 7445HS with six cores, twelve logical CPUs, 15.2 GiB RAM, and a Samsung MZVL8512HFLU NVMe SSD, running Windows 11 Home build 26200 on NTFS with `GOMAXPROCS=12`. Every campaign recorded the Go toolchain as go1.26.3. Except for table-count variants, fixtures have 10,000 keys in four disjoint tables, 128-byte values, and uniform lookups with target hit ratio 0.95. Trials use 2 s reader-only warmup, a 10 s window, and 200 ms cooldown. Delay is injected before each point-read `ReadAt`, including writer validation.

The seven campaigns are the initial campaign, the intervention, compaction, table count, paced load, settled, and the follow-up. The settled campaign also reran selected earlier cells, after a pause, as sentinels. Mixed uses a free-running writer and a 1 MiB memtable. Publication releases twenty pre-frozen memtables every 0.5 s, and Churn uses a 16 KiB memtable with frequent flushing. Mixed and Publication also include a legacy-scan cell, an earlier full-scan reader, reported only in the extended report; it is included in H2's ten and fourteen tests. Paced fixes intended write arrivals at fractions f of earlier Lease capacity with closed or open readers. Compaction variants merge all active tables on a fixed timer, and table-count variants use disjoint ranges up to 256 tables. Mixed and the intervention use $r \in \{4, 16\}$ and $d \in \{0, 20\,\mu\text{s}, 200\,\mu\text{s}, 2\,\text{ms}\}$. Publication uses $r = 16$ and $d \in \{0, 200\,\mu\text{s}, 2\,\text{ms}\}$; Churn uses $r \in \{1, 4, 16\}$ at zero delay. Compaction uses $r \in \{4, 16\}$ and $d \in \{0, 200\,\mu\text{s}, 2\,\text{ms}\}$. Table count uses $r \in \{4, 16\}$ at zero delay, with 4, 16, 64, or 256 tables. Paced uses $r = 16$, $d \in \{0, 200\,\mu\text{s}, 2\,\text{ms}\}$, and $f \in \{0.2, 0.8\}$: 183/732, 155/619, and 72/286 writes/s at the three delays, each pair ordered $f = 0.2/0.8$. The settled campaign adds 700 writes/s at 200 μs.

The follow-up fixes $r = 16$, one paced writer, closed readers, and zero delay at 183, 366, 549, and 732 writes/s with a 16 MiB memtable, so no flush falls in a window. It adds one 732 writes/s cell with fsync off, one with a 1 MiB memtable reported in the extended report, and a rerun of one Mixed cell ($r = 4$, zero delay, all three designs) that stores full read-latency histograms. H-F4 uses that last cell.

Pairing holds seeds and operation streams fixed within each cell-round unit, with randomized interleaving. Most campaigns use ten matched triplets per cell; the intervention uses five four-run units. Throughput counts in-window completions per measured window. Service time runs from issue to completion and response time from intended arrival; a trial below 0.95 completed/offered writes is flagged saturated, and completion tails under overload are censored from below. We write r for readers, d for delay, W, R for write and read throughput, $S = W_{\text{CL}}/W_{\text{L}}$, and $q = R/(rW)$.

Equivalence uses paired log ratios and the margin $[0.90, 1/0.90]$, chosen by the author, with Holm-adjusted TOST [24]–[26]. A CI wholly beyond the margin is not equivalent, and other results are inconclusive. Trend tests use exact Page tests [23].

Seven protocols were frozen before campaign trials [27]. Table II lists the main predictions; the extended report [33] lists all 24 outcomes. The first five campaigns ran back to back without a deliberate pause; the settled and follow-up campaigns added a pause. All 2,499 retained trials succeeded on the first attempt.

IV. WHAT HOLDING THE LOCK COSTS

CoarseLock's write penalty increased with measured read-lock hold (Fig. 2). At about 34 μs and four readers it completed 7.4% fewer writes than Lease, ratio 0.926 with 95% CI $[0.881, 0.974]$; around 163–173 μs losses were 24–30%, and around 1.9 ms they were 64–69%. H1 supported a decreasing ratio across doses at both reader counts, with Page's $L = 294$, exact one-sided $p = 7.86 \times 10^{-9}$, and ten paired blocks each.

TABLE II
MAIN PRE-REGISTERED PREDICTIONS AND OUTCOMES. H-S IS A PROTOCOL CHECK. ALL 24 OUTCOMES ARE IN THE EXTENDED REPORT.

Prediction	In plain words	Outcome	Section
H1	CoarseLock/Lease write ratio falls as delay rises	Supported	IV
H2	Lease/Snapshot throughput and publication waits match the margin	Partly supported (waits not established)	VI
Intervention P1–P3	Reads per reader per commit follow acquisition count; removing one acquisition lowers the read ratio; cycle errors fit	Supported	V
H-C	Lease/Snapshot throughput matches with compaction on	Supported	VI
H-T2	Equivalence at four tables; read-only separation at 256	Supported	VI
H-P1	CoarseLock/Lease write response p99 ratio rises with delay	Failed	IV
H-P2	Below-capacity write throughput matches the margin	Supported	IV
H-P3	Higher-load writes match at zero delay and separate at higher doses	Partly supported; failed at 200 μs	IV
H-M1–H-M3	Median response ratio rises with delay; both CoarseLock waits fit the delay band	Supported	IV
H-P4, H-R, H-F3	Lease/Snapshot reads match across paced write rates, including the cell left open	H-P4, H-F3 partly supported; H-R failed	VI
H-W	CoarseLock/Lease write throughput separates at 700 writes/s	Supported	IV
R-check	Rechecked metrics and classes; rerun if the rule triggers	No rerun triggered; five metrics inconclusive	VIII
H-S	Protocol check: median largest fsync below a ceiling raised from 25 to 40 ms	Supported at the raised threshold	VIII
H-F1	Reads scale with free share across write rates, slope in $[0.80, 1.25]$	Failed	VII
H-F2	Reads rise as write rate falls	Supported	VII
H-F4	CoarseLock read latency has fast and blocked groups with a near-empty gap	Supported	V

The point estimates were not strictly monotonic: at sixteen readers S rose from 0.937 to 0.944 between zero and 20 μs. Below about 40 μs the largest point loss, 7.4%, is not a confidence bound: the 20 μs intervals were $[0.881, 0.974]$ and $[0.887, 1.004]$, allowing larger losses and, for sixteen readers, no loss. Lease's mean commit cycle at four readers also rose from 1.07 ms at zero delay to 2.79 ms at 2 ms in an exploratory comparison, because validation reads delayed blocks under the exclusive lock.

Below capacity, write throughput matched Lease while median write response rose with delay. H-P2 established equivalence in 6/6 paced cells. H-M1 held in both reader modes with maximum Page's $L = 140$, exact $p = 1.65\times10^{-8}$, Holm-adjusted $p = 3.31 \times 10^{-8}$, and every round in the predicted order. Response p50 ratios at zero delay, 200 μs, and 2 ms were 1.080, 1.431, 2.663 with closed readers and 1.015, 1.332, 2.414 with open readers, and H-M2 held in all four higher-dose tests (Table III). At zero delay there was no prediction; the ratios 1.080 [1.067, 1.092] and 1.015 [0.998, 1.032] had 90% CIs inside the margin. Both lock waits behind the median penalty were measured, supporting H-M3: at 2 ms with closed readers CoarseLock's first wait was 1.952 [1.911, 1.993] ms and its flush check 2.105 [2.099, 2.112] ms, with 95% CIs, against 0.0003 and 0.027 ms for Lease.

Past capacity the comparison changes. H-P3 failed at 200 μs because its offered rate placed capacity next to the equivalence boundary: it met 4/6 predictions, with 200 μs inconclusive in both modes, 0.896 [0.866, 0.927] and 0.920 [0.898, 0.942]. The flaw was visible before freezing, since earlier capacity divided by offered rate was $545.8/619 = 0.882$, already next to the 0.90 margin. H-W repaired the design at 700 writes/s: ratios 0.792 [0.775, 0.808] and 0.840 [0.818, 0.863], with 90% CIs, were both not equivalent.

H-P1 failed. At $f = 0.8$ the p99 trend tests were significant, with adjusted $p = 1.4 \times 10^{-6}$ for closed and 1.0×10^{-5} for open readers, but every round had a censored component, so they detect backlog and not true p99s. At $f = 0.2$, where the ratio was predicted to rise below capacity, it did not: adjusted $p = 0.291$ with closed and 0.093 with open readers. The p99 was unusable as a trend metric, since a few stalls, with individual write-service maxima of 32–103 ms, decide the p99 of about 1830 writes.

V. ADMISSION COUPLING

A. Reads follow exclusive acquisitions

With a saturated writer CoarseLock read more than Lease through the 200 μs dose and less at 2 ms, a crossing that was unpredicted. If each exclusive acquisition admits approximately one read per reader, then

$$R_{\mathrm{CL}} \simeq k\, r\, W_{\mathrm{CL}}, \qquad k = 2 \text{ on the default path.} \tag{1}$$

Individual trials in the eleven Mixed and Churn cells with added or zero delay had $q = 2.002$–2.015 for CoarseLock, despite changing delay and reader count. Lease and Snapshot readers do I/O outside the lock and can miss the short second admission: Lease's cell mean q fell from about 1.7 at zero delay to about 1.3 at 20 and 200 μs and about one at 2 ms. An exploratory cycle check predicts CoarseLock's commit cycle as Lease's cycle plus two mean first-acquisition waits; its errors were -3.6% to $+10.6\%$ in the eight Mixed cells with added or zero delay. Paced counts also replicated the coupling out of sample: with closed readers at $f = 0.8$ and 200 μs CoarseLock read 17,560 per second against $2rW = 17{,}511$.

By definition of q, $R_{\mathrm{CL}}/R_{\mathrm{L}} = (q_{\mathrm{CL}}/q_{\mathrm{L}})\, S$; for the stored cell summaries the product reproduces the read ratio to within 0.0001. CoarseLock read more than Lease in six Mixed cells (Table IV). Arithmetically, the gain comes from CoarseLock's higher q, and its lower write throughput shrinks it. In two of the six cells the write-loss CI includes one while the read gain's excludes it: at four readers and zero delay S was 0.988 [0.896, 1.089] with reads 1.193 [1.089, 1.308], and at sixteen readers and 20 μs S was 0.944 [0.887, 1.004] with reads 1.432 [1.350, 1.519].

B. Changing acquisition count tests the account

The pre-registered intervention changed k below the flush threshold. An experiment-only flag checks the threshold inside

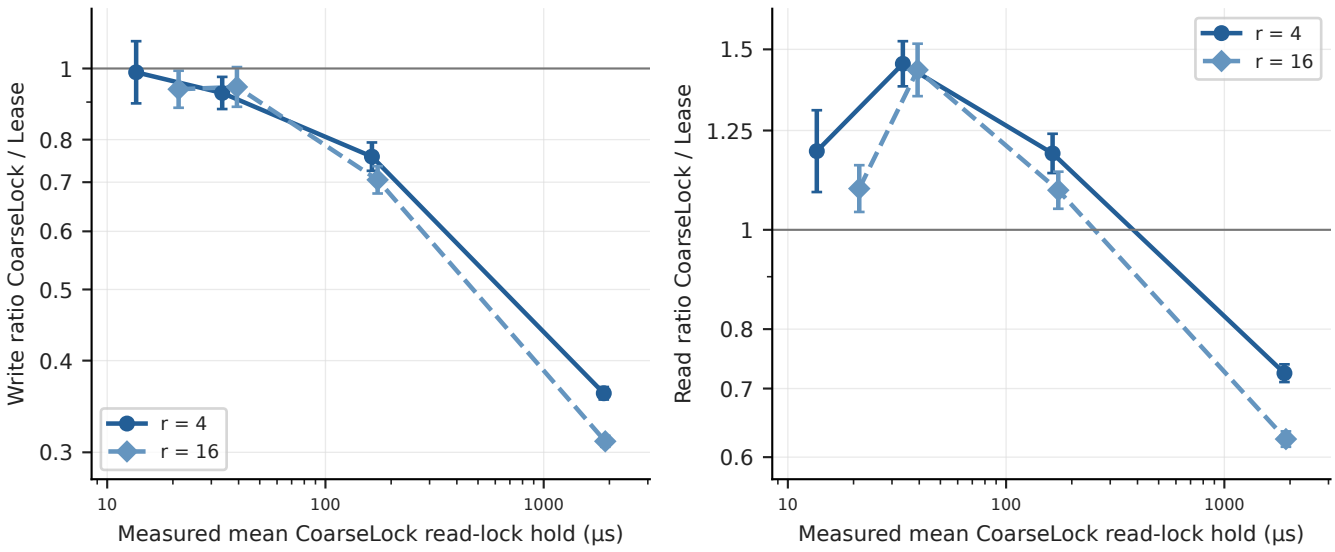


Fig. 2. Mixed workload: paired CoarseLock/Lease ratios against CoarseLock's arithmetic mean trial read-lock hold, at four injected doses. Bars are 95% paired log-t CIs. Measured hold is descriptive, not a separately randomized treatment.

TABLE III

SETTLED CAMPAIGN, TEN ROUNDS PER CELL. THE WRITE RESPONSE P50 RATIO IS THE PAIRED GEOMETRIC COARSELOCK/LEASE MEAN WITH ITS 90% CI; PHASES ARE ARITHMETIC MEANS OF PER-TRIAL P50 IN MS; THE LAST COLUMN IS THE CELL MEDIAN OF THE TRIAL LARGEST FSYNC IN MS. AT 700 WRITES/S COARSELOCK IS PAST CAPACITY, SO ITS RESPONSE P50 IS A COMPLETION-ONLY BACKLOG FIGURE AND THOSE RATIOS ARE CENSORED, NOT TRUE LATENCY RATIOS.

			CoarseLock/Lease	CoarseLock		Lease		Largest
Delay	Writes/s	Readers	Response p50 [90% CI]	Lock wait	Flush check	Lock wait	Flush check	fsync
0	183	closed	1.080 [1.067, 1.092]	0.075	0.074	0.001	0.047	21.8
0	183	open	1.015 [0.998, 1.032]	0.000	0.066	0.000	0.062	11.4
200 µs	155	closed	1.431 [1.398, 1.464]	0.229	0.296	0.000	0.033	9.8
200 µs	155	open	1.332 [1.325, 1.340]	0.137	0.264	0.000	0.028	8.9
2 ms	72	closed	2.663 [2.640, 2.687]	1.952	2.105	0.000	0.027	6.9
2 ms	72	open	2.414 [2.372, 2.456]	1.846	2.088	0.000	0.018	7.2
200 µs	700	closed	599.389 [195.644, 1836.337]	0.268	0.281	0.001	0.032	31.5
200 µs	700	open	206.265 [54.672, 778.187]	0.226	0.262	0.000	0.031	31.5
0	732	closed	1.114 [1.009, 1.231]	0.060	0.065	0.003	0.042	31.5

the first exclusive hold and skips the second acquisition when no flush is needed; validation and WAL work remain. Five paired four-run units per cell compare CoarseLock and Lease on both paths. All P1–P3 criteria held (Fig. 3). P1: individual CoarseLock trial q ranged 2.002–2.013 for $k = 2$ and 1.001–1.006 for $k = 1$. P2: the six applicable ratio-of-read-ratios CIs lay below one; at four readers and 20 μs it was 0.659, 95% CI $[0.595, 0.730]$. P3: cycle errors were -3.7% to $+7.5\%$ with two acquisitions and -2.0% to $+8.4\%$ with one, and all four higher-dose write-improvement CIs exceeded one.

With one acquisition, the CoarseLock/Lease read ratios at zero and 20 μs were 0.963 [0.897, 1.033] and 0.975 [0.882, 1.077] at four readers and 0.987 [0.895, 1.089] and 0.983 [0.897, 1.077] at sixteen, all with 95% CIs including one. With one acquisition CoarseLock's read ratio was below one as a point estimate at every dose and reader count (0.864, 0.848, 0.630, and 0.559 at 200 μs and 2 ms), with CIs below one at those two doses. The read gain at injected delays up to 20 μs therefore depended on the redundant second acquisition. Removing it raised CoarseLock writes, with CIs above one, at 200 μs and 2 ms and at sixteen readers with zero delay; the CIs included one at four readers with zero delay and 20 μs and at sixteen readers with 20 μs. For Lease, write ratios were 0.970–1.044, all with CIs including one, while its reads fell to 0.608–0.850 of the two-acquisition level. Apart from P3's higher-dose write gains, these comparisons are exploratory, with five pairs.

C. Equal write rates and the blocked group

At write rates we could match, the read gain was absent in a descriptive comparison. With closed readers, $r = 16$, $f = 0.2$, and no flushes, CoarseLock and Lease achieved matched write rates at zero delay, 200 μs, and 2 ms, and the read ratios were 0.967 [0.912, 1.025], 0.921 [0.889, 0.954], 0.720 [0.711, 0.730], with 95% CIs. At $f = 0.8$ only the zero-delay cell had near-equal writes, with read ratio 0.776 [0.666, 0.904]. The 200 μs and 2 ms cells at $f = 0.8$ do not have equal writes, since CoarseLock completed 0.884 and 0.388 of its offered

TABLE IV

THE SIX MIXED CELLS WHERE COARSELOCK READ MORE THAN LEASE AT THE DEFAULT COMMIT. q VALUES ARE ARITHMETIC MEANS; S AND THE READ RATIO ARE PAIRED GEOMETRIC RATIOS WITH 95% t CIS.

r	Delay	q_{CL}	q_L	q_{CL}/q_L	S [95% CI]	R_{CL}/R_L [95% CI]
4	0	2.009	1.664	1.208	0.988 [0.896, 1.089]	1.193 [1.089, 1.308]
4	20 μs	2.003	1.278	1.568	0.926 [0.881, 0.974]	1.452 [1.380, 1.528]
4	200 μs	2.004	1.280	1.565	0.758 [0.725, 0.793]	1.187 [1.136, 1.241]
16	0	2.010	1.717	1.171	0.937 [0.884, 0.993]	1.097 [1.041, 1.156]
16	20 μs	2.008	1.323	1.518	0.944 [0.887, 1.004]	1.432 [1.350, 1.519]
16	200 μs	2.006	1.295	1.550	0.705 [0.676, 0.736]	1.093 [1.048, 1.140]

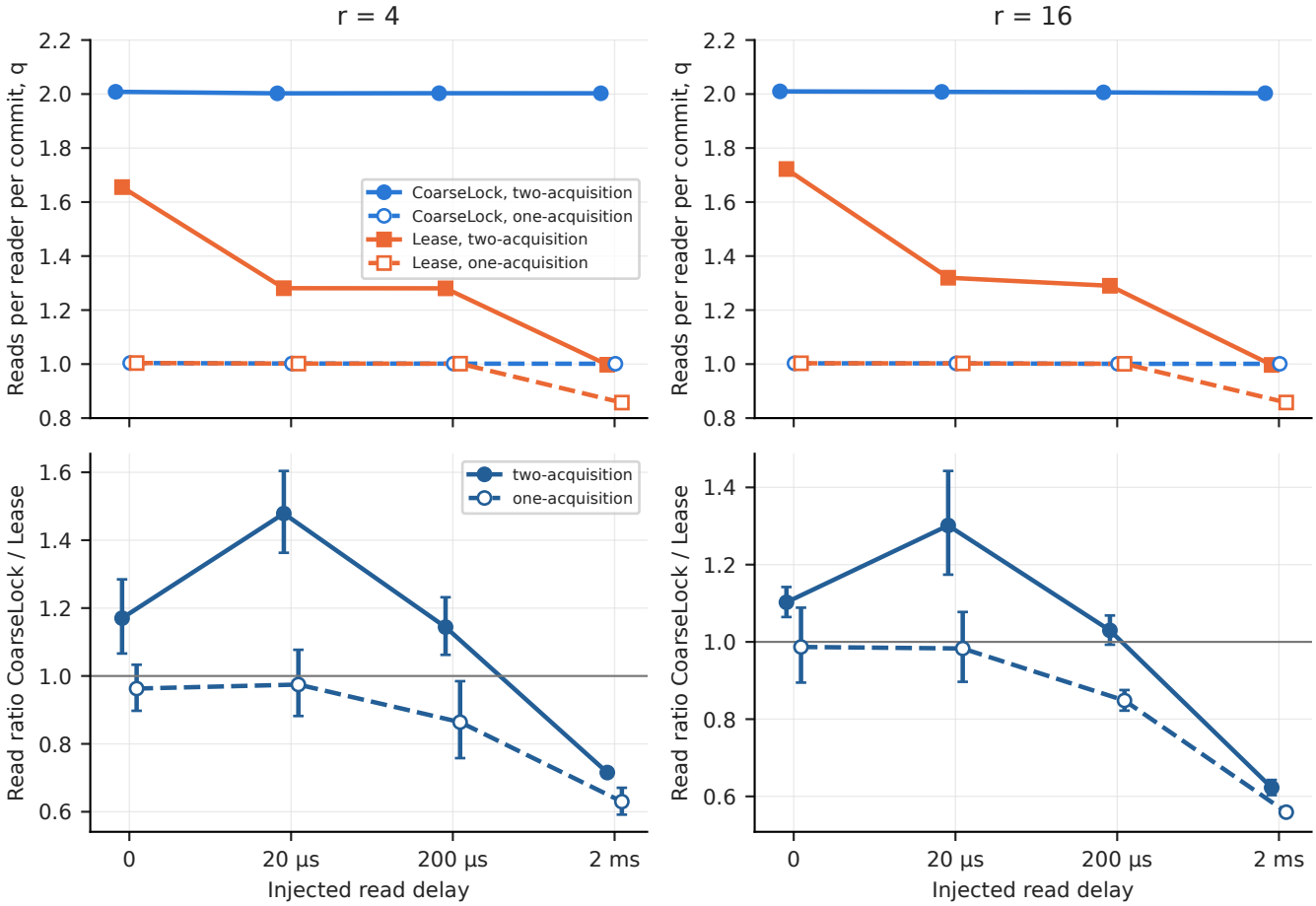


Fig. 3. Intervention, five paired four-run units per cell. Top: arithmetic mean reads per reader per commit. Bottom: paired geometric CoarseLock/Lease read ratios with 95% log-t CIs. Solid filled markers denote two-acquisition commits; dashed hollow markers denote one-acquisition commits below threshold.

writes, so they are not equal-rate evidence. No paced cell used 20 μs, where the saturated gain peaked. Below capacity the writer is not continually waiting, so this compares regimes.

H-F4 held in the follow-up Mixed cell: CoarseLock reads split into a fast group below 200 μs (55.5 [55.2, 55.8]%, 95% CI) and a slow group at or above 500 μs (44.4 [44.1, 44.7]%), with 0.11 [0.09, 0.13]% between them. The slow group's median, 854–877 μs, is a little below the 0.98–1.11 ms hold per commit traced at paced rates. This cell carries no trace, so blocking behind a commit is inferred from that similarity and not measured.

VI. WHEN DO LEASE AND SNAPSHOT DIFFER?

Lease and Snapshot met H2's throughput equivalence in every Mixed and Publication cell: all ten write and fourteen read tests passed after Holm correction, and publication at zero delay still had read ratio 0.970, 90% CI $[0.962, 0.978]$, a roughly 3% difference within the margin. In an exploratory tighter-margin check, 6 of ten write and 11 of fourteen read paired-t CIs lay inside $[0.95, 1/0.95]$. Publication waits were not established as equivalent: the zero-delay Lease/Snapshot ratio of publication lock-wait p99 was 3.11, 90% CI $[1.54, 6.30]$, not equivalent, and the other three cells were inconclusive.

With compaction on, all twelve primary tests met H-C; the closest write comparison was 0.949, 90% CI $[0.909, 0.991]$.

Read-only load separated them as table count grew. H-T2 held: equivalence at four tables and read-only non-equivalence at 256 tables for both reader counts. Read-only ratios across 4, 16, 64, and 256 tables were 0.975, 0.956, 0.918, 0.712 at four readers and 0.989, 0.962, 0.851, 0.663 at sixteen. At sixteen readers, 64 tables already gave 0.851, 90% CI $[0.837, 0.866]$, not equivalent. At 256 tables the ratios were 0.712 at four readers and 0.663, 90% CI $[0.660, 0.666]$, at sixteen, both not equivalent; the roughly 34% fewer reads applies to sixteen readers only. With a free writer all sixteen applicable tests were equivalent, with ratios 0.943–1.045 (Fig. 4).

Lease's read path does work per active table. Read-only mean holds grew with table count, spanning 326–3370 ns at four readers and 393–8512 ns at sixteen, while Snapshot stayed below 1 μs. Lease leases every active table on every read, so this may reflect this Lease and not leasing in general. A separate microbenchmark of capture and release cost Lease 27–974 ns at one goroutine against Snapshot's 21 ns, while publication cost Lease 104–128 ns and Snapshot 654–3200 ns; that cheaper publication was not visible end to end.

Read equivalence was not established at high write rates. At zero delay and $f = 0.8$ H-P4 left the comparison inconclusive, and on fresh data H-R failed: writes were equivalent, 1.001 [0.994, 1.008], but reads were inconclusive, 0.995 [0.846, 1.171], adjusted $p = 0.142$. In the follow-up, H-F3 found Lease/Snapshot reads equivalent at 183 and 366 writes/s, inconclusive at 549, 0.983 [0.871, 1.108], and inconclusive at 732, 1.023 [0.839, 1.246], where we made no prediction (all 90% CIs). The log-ratio SD grew from 0.023 at 183 writes/s to 0.341 at 732. The estimates are close to one, but at high write rates the comparison lacks the precision to show equivalence.

Lease's commit-wait p99 exceeded Snapshot's in an exploratory comparison from a back-to-back campaign. CoarseLock and Lease open compaction inputs under the exclusive lock and Snapshot opens them outside; this asymmetry is known. The rechecked ratio moved from 1.43 to 2.40 and was classed inconclusive, and the cause is untested.

VII. The Writer's Limit

The writer's own hold limits both replacements. Hold share is the share of the window spent in a commit's first exclusive hold (validation, WAL write, WAL sync, and apply); it excludes the short flush-check hold and lock waits. Free share is one minus hold share. In the follow-up, hold share rose from about 20% at 183 writes/s to about 72% at 732, and reads fell from about 370,000 to about 166,000 per second. The hold per commit stayed near one millisecond, 0.98–1.11 ms across rates and designs, and WAL sync took 94.1–96.3% of the exclusive hold. Both designs take `Mu.RLock` to search memory, so the exclusive hold blocks their readers; LevelDB releases its mutex around log sync [3].

H-F2 held, with Page's $L = 299$ and adjusted $p < 0.001$ for both designs. H-F1 failed. Across the four rates the slope of log reads on log free share was 0.768 [0.731, 0.806] for Lease and 0.779 [0.745, 0.813] for Snapshot, with 90% CIs below the predicted band $[0.80, 1.25]$ and excluding 1 (Fig. 5). Reads fall as the hold grows but not in proportion to free share. One candidate, which we have not tested, is that read cost falls as write rate rises. Writes in these cells use the read keys and stay in the memtable, and a read found in memory skips the SSTable search. The share of reads served from memory rose from about 0.08 at 183 writes/s to about 0.29 at 732, and reads per unit of free share rose from about 460,000 to about 600,000 per second. Write rate and memory share move together here, so these cells cannot separate this from a rival in which fsync costs readers more than its hold at low write rates. A frozen protocol that would separate them has not been run. In one exploratory comparison at 732 writes/s, turning fsync off raised reads 3.414 and 3.530 times for Lease and Snapshot and free share 3.470 and 3.600 times, so read gain over free-share gain was 0.984 [0.963, 1.005] and 0.981 [0.950, 1.012] (90% paired-t CIs). Fsync off gives up durability, so this does not estimate what a commit-path change would recover. Memory share was similar in those two cells, and read gain tracked free-share gain, which fits a factor tied to write rate without establishing one.

VIII. Trial Spacing

Trials started immediately after another had a largest fsync near 55 ms. In 9/9 same-specification pairs the largest fsync was higher back to back, with median 54.9 ms, range 45.7–73.0, against 8.0 ms, range 5.7–17.0, after 30 s pauses, with nonoverlapping ranges (exploratory diagnosis). The pause removed that band, and the cause is unknown. Calibration pauses of at least 15 s removed the band above about 45 ms in 18/18 trials, and the settled protocol froze a 20 s pause. A separate burst near 31 ms was not removed by pausing and was uncontrolled, and its cause is unknown; it is why some H-S medians reach 31 ms.

H-S is a protocol check. We raised its threshold from 25 to 40 ms after seeing calibration trials and before any campaign trial. Three of 18 design-level medians (31.4–31.5 ms) exceed 25 ms, so H-S would have failed at the draft threshold; the medians spanned 6.9–31.5 ms.

The sentinel recheck, R-check, triggered no rerun: of 22 metrics, 17 were unchanged, 5 inconclusive, and 0 changed, and no published class was contradicted. Tails moved: at 2 ms, CoarseLock/Lease response p99 ratios fell from 4.04 to 1.85 with closed readers and from 4.98 to 1.64 with open readers. We recommend pausing between storage trials, reporting the gap, and checking results against a paused sentinel.

IX. Practical Guidance

1) Measure lock-held lookup time and split commit time into validation, synchronization, and lock waits.
2) Compare at the intended write rate: below capacity, we saw gains in median response and not throughput.
3) Count exclusive acquisitions per commit. Do not read a saturated-writer read gain as an argument against the change.

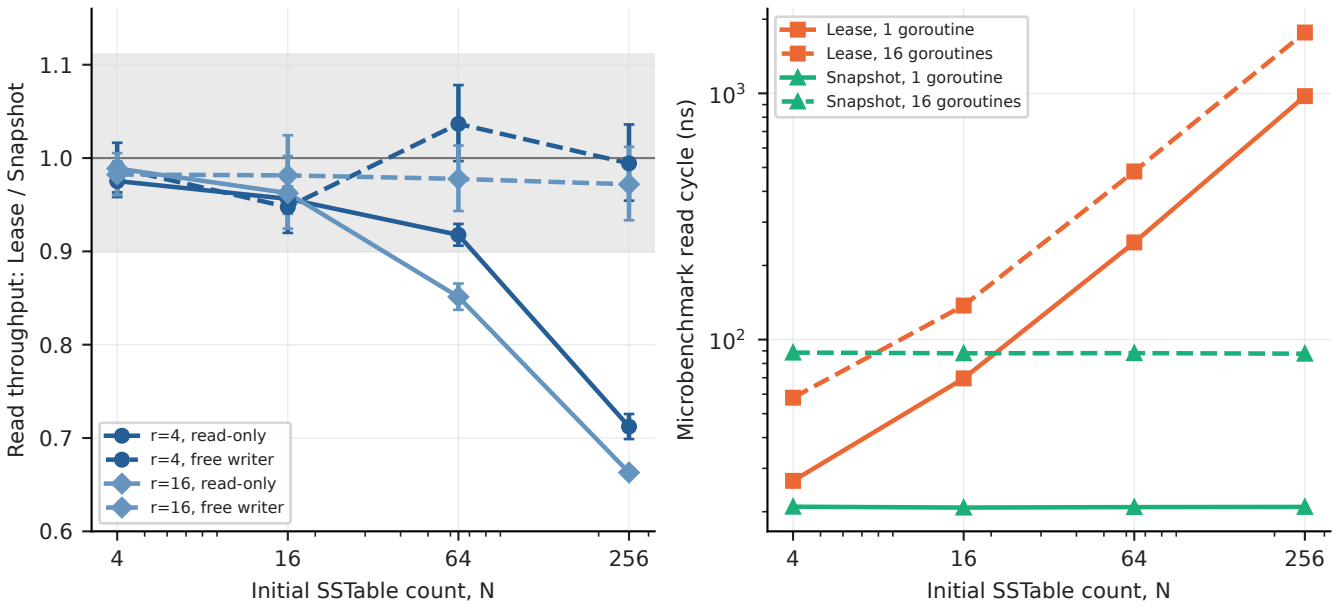


Fig. 4. Table-count campaign and separate lifecycle microbenchmark. Left: Lease/Snapshot read ratios with 90% paired log-t CIs; solid lines are read-only and dashed lines have one free writer. Right: arithmetic means of three benchmark counts, with no campaign CI. The benchmark isolates capture/release and excludes search and I/O.

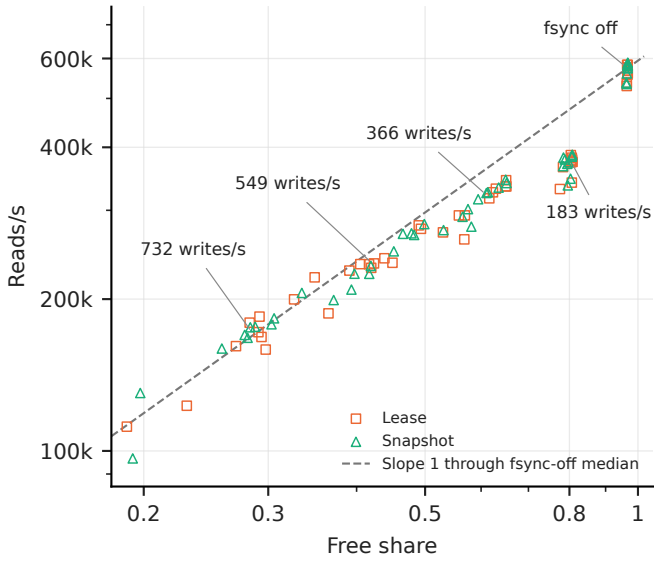


Fig. 5. Reads against the share of the window outside the writer's exclusive hold, every Lease and Snapshot trial at four write rates and with fsync off, zero delay, closed readers, log axes. The dashed line has slope 1 through median fsync-off capacity. H-F1 predicted a slope in $[0.80, 1.25]$ across the four rates and failed.

4) Test the intended table count: no throughput comparison favoured Lease over Snapshot beyond the declared margin, Snapshot read more under read-only load at high table counts, and complexity was not measured.
5) Measure the writer's hold share and look at the commit path; we did not test group commit.
6) Space trials and check against a paused sentinel.

X. Threats to Validity

Injected delay controls read cost rather than reproducing a device. The small dataset was likely page-cache resident, with no block cache, and the delay treatment changes from yielding spin to sleep. Sixteen readers run on twelve logical CPUs. Unissued write ticks have no latency observation, so completion tails are censored. The merge-all compaction timer differs from production policies. The first five campaigns ran back to back, and the sentinels confound the pause with date, binary, tracing, and seeds. The untested read-cost candidate in Section VII remains confounded with write rate. The equivalence margin is the author's judgment, and Holm adjustment applies within declared families only. The intervention has five pairs and no multiplicity correction. Hypotheses arose from earlier outcomes, and there is a sole experimenter and no independent replication. Host state remains a source of variation despite pairing and randomized order [29], and fsync bursts near 31 ms were uncontrolled. Results come from one engine on one Windows host with at most one writer; the implementations do not reproduce LevelDB, RocksDB, or Pebble, and the coupling equation describes this Go lock. Free-share scaling gives a direction and not a calibrated model: reads were about 19% below it at 183 writes/s and about 5% above it at 732. The extended report [33] lists exposures and deviations.

XI. Related Work

LSM concurrency work scales other engine components: cLSM uses non-blocking structures [7], FloDB scales the

memory component [8], SILK schedules internal I/O [9], and performance-stability work studies write stalls [10]; RocksDB's history describes production priorities [6]. Reclamation research compares hazard pointers with epoch and quiescent-state methods [19], [20], and reader-writer lock work addresses admission and reader cost [13]–[17]. Group commit and early lock release address writer work that the reader-side replacements retain [21], [22]. Benchmarking guidance covers host-state pitfalls and open versus closed load [28], [30]–[32]. These works address other components, locks in isolation, or method. This study measures how a reader-side lifetime fix interacts with a writer's exclusive hold and with admission in one engine.

XII. Conclusion

A reader-side lifetime fix helps when shortening the read hold relieves writer waiting at the intended load, and the writer's own critical section limits the gain. Each exclusive acquisition admits a round of readers, so the coarse lock's read gain at short lookups depends on the redundant second acquisition. Below capacity the cost appears in median response, and reads fall as the writer's hold grows. Lease and Snapshot match on throughput across publication and compaction, and read-only load at high table counts separates them. These results cover one engine on one Windows host, at most one writer, injected delay, and WAL fsync under the exclusive lock.

Acknowledgment

Anthropic's Claude and OpenAI's Codex were used to assist with engineering, analysis scripts and drafting of text. The author reviewed the results and is responsible for the content.

Artifact Availability

The repository is https://github.com/itsramhere/selfdbms, with campaign records on branches `research/x-readcost` and `research/x-phase3`. Evidence is indexed by `paper/CLAIM_EVIDENCE_MAP_X.md`; the reproducibility appendix is `paper/REPRODUCIBILITY_APPENDIX.md`, and paper names map to repository identifiers in `paper/REPOSITORY_IDENTIFIERS.md`. The extended report, `paper_extended.pdf`, gives the full cells, protocols, and execution history.

References

[1] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.

[2] C. Luo and M. J. Carey, "LSM-based storage techniques: a survey," *The VLDB Journal*, vol. 29, no. 1, pp. 393–418, 2020.

[3] S. Ghemawat and J. Dean, "LevelDB." [Online]. Available: https://github.com/google/leveldb

[4] Meta Platforms, "RocksDB." [Online]. Available: https://github.com/facebook/rocksdb

[5] Cockroach Labs, "Pebble." [Online]. Available: https://github.com/cockroachdb/pebble

[6] S. Dong, A. Kryczka, Y. Jin, and M. Stumm, "RocksDB: Evolution of development priorities in a key-value store serving large-scale applications," *ACM Trans. Storage*, vol. 17, no. 4, Art. 26, 2021.

[7] G. Golan-Gueta, E. Bortnikov, E. Hillel, and I. Keidar, "Scaling concurrent log-structured data stores," in *Proc. EuroSys*, 2015, Art. 32.

[8] O. Balmau, R. Guerraoui, V. Trigonakis, and I. Zablotchi, "FloDB: Unlocking memory in persistent key-value stores," in *Proc. EuroSys*, 2017, pp. 80–94.

[9] O. Balmau, F. Dinu, W. Zwaenepoel, K. Gupta, R. Chandhiramoorthi, and D. Didona, "SILK: Preventing latency spikes in log-structured merge key-value stores," in *Proc. USENIX ATC*, 2019, pp. 753–766.

[10] C. Luo and M. J. Carey, "On performance stability in LSM-based storage systems," *Proc. VLDB Endow.*, vol. 13, no. 4, pp. 449–462, 2019.

[11] W. Pugh, "Skip lists: A probabilistic alternative to balanced trees," *Commun. ACM*, vol. 33, no. 6, pp. 668–676, 1990.

[12] P. J. Courtois, F. Heymans, and D. L. Parnas, "Concurrent control with 'readers' and 'writers'," *Commun. ACM*, vol. 14, no. 10, pp. 667–668, 1971.

[13] J. M. Mellor-Crummey and M. L. Scott, "Scalable reader-writer synchronization for shared-memory multiprocessors," in *Proc. PPoPP*, 1991, pp. 106–113.

[14] B. B. Brandenburg and J. H. Anderson, "Reader-writer synchronization for shared-memory multiprocessor real-time systems," in *Proc. ECRTS*, 2009, pp. 184–193.

[15] I. Calciu, D. Dice, Y. Lev, V. Luchangco, V. J. Marathe, and N. Shavit, "NUMA-aware reader-writer locks," in *Proc. PPoPP*, 2013, pp. 157–166.

[16] R. Liu, H. Zhang, and H. Chen, "Scalable read-mostly synchronization using passive reader-writer locks," in *Proc. USENIX ATC*, 2014, pp. 219–230.

[17] D. Dice and A. Kogan, "BRAVO: Biased locking for reader-writer locks," in *Proc. USENIX ATC*, 2019, pp. 315–328.

[18] The Go Authors, "sync package: RWMutex." [Online]. Available: https://pkg.go.dev/sync#RWMutex

[19] M. M. Michael, "Hazard pointers: Safe memory reclamation for lock-free objects," *IEEE Trans. Parallel Distrib. Syst.*, vol. 15, no. 6, pp. 491–504, 2004.

[20] T. E. Hart, P. E. McKenney, A. D. Brown, and J. Walpole, "Performance of memory reclamation for lockless synchronization," *J. Parallel Distrib. Comput.*, vol. 67, no. 12, pp. 1270–1285, 2007.

[21] D. J. DeWitt, R. H. Katz, F. Olken, L. D. Shapiro, M. R. Stonebraker, and D. A. Wood, "Implementation techniques for main memory database systems," in *Proc. ACM SIGMOD*, 1984, pp. 1–8.

[22] R. Johnson, I. Pandis, R. Stoica, M. Athanassoulis, and A. Ailamaki, "Aether: A scalable approach to logging," *Proc. VLDB Endow.*, vol. 3, no. 1–2, pp. 681–692, 2010.

[23] E. B. Page, "Ordered hypotheses for multiple treatments: A significance test for linear ranks," *J. Amer. Statist. Assoc.*, vol. 58, no. 301, pp. 216–230, 1963.

[24] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *J. Pharmacokinet. Biopharm.*, vol. 15, no. 6, pp. 657–680, 1987.

[25] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[26] D. Lakens, "Equivalence tests: A practical primer for t tests, correlations, and meta-analyses," *Soc. Psychol. Personal. Sci.*, vol. 8, no. 4, pp. 355–362, 2017.

[27] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. Natl. Acad. Sci. USA*, vol. 115, no. 11, pp. 2600–2606, 2018.

[28] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. OOPSLA*, 2007, pp. 57–76.

[29] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!," in *Proc. ASPLOS*, 2009, pp. 265–276.

[30] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems," in *Proc. SC*, 2015, pp. 1–12.

[31] A. Traeger, E. Zadok, N. Joukov, and C. P. Wright, "A nine year study of file system and storage benchmarking," *ACM Trans. Storage*, vol. 4, no. 2, Art. 5, 2008.

[32] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proc. NSDI*, 2006.

[33] V V Ramarathinam Iyer, "When does a reader-side lifetime fix help? Admission coupling in an LSM engine (extended report)," 2026. [Online]. Available: `paper_extended.pdf` in the artifact repository, https://github.com/itsramhere/selfdbms